

Avalon-MM SD Card Controller IP Core

User Guide

IP core	<code>altera_avalon_mm_sdcard_controller</code>
Core version	1.0
Document version	1.0
Date	September 2026
Target	Intel FPGA / Quartus Prime, Platform Designer

Contents

1. About this core	4
1.1 Features	4
1.2 What it is for	4
1.3 Status	4
2. Getting started	6
2.1 Adding the component	6
2.2 Connecting it	6
2.3 Pin assignment	6
2.4 A first transfer	7
3. Functional description	8
3.1 The shape of an operation	8
3.2 Multi-block transfers	10
3.3 Timeouts	10
3.4 Soft resets	10
3.5 CRC	10
4. Parameters	11
4.1 On MAX_BLOCK_BYTES	11
4.2 On USE_DMA	11
5. Registers	12
5.1 CTRL (0x00)	12
5.2 STATUS (0x04)	13
5.3 IRQ_STATUS (0x0C) and IRQ_ENABLE (0x08)	13
5.4 CMD (0x1C)	14
5.5 ERR_INFO (0x3C)	14
6. Signals	15
6.1 Clock and reset	15
6.2 csr — Avalon-MM slave	15
6.3 m0 — Avalon-MM master (USE_DMA = 1)	15
6.4 sd — conduit	16
7. Software	17
7.1 Capacity, and why the CSD parse is separately tested	17
7.2 Card changes	18
7.3 Retries	18
7.4 Buffers and PIO	18
7.5 Non-blocking transfers	18
7.6 FatFs	19

8. Performance	20
8.1 Measured	20
8.2 Where it comes from	20
8.3 What the DMA is and is not for	20
9. Verification	21
9.1 The configuration sweep	21
9.2 Assertions, and proving they are alive	21
9.3 The driver against the RTL	22
9.4 Questa	22
10. Limitations and known gaps	23
11. Revision history	24

1. About this core

1.1 Features

- SD, SDHC and SDXC cards in **SPI mode** (Physical Layer Simplified Specification v4.10, chapter 7)
- Hardware link layer: command framing, CRC7 generation, CRC16 generation and checking, token recognition, data-response decoding, busy detection
- **Multi-block streaming in hardware**, including the automatic CMD12 stop for multi-block reads
- **Optional bursting DMA** (`m0`) that moves block data to and from memory without CPU involvement — or PIO through a register window when you have no master to spare
- Run-time SPI clock divider covering the 400 kHz identification rate and full speed from the same build
- Bounded waits on every phase, reporting **which** phase failed
- Split soft resets: reset the data path without losing card identification
- Nios II HAL driver with `alt_sdcard_*` API: identifies the card on first use, notices card changes, retries CRC errors, and with the DMA can start a transfer and return, with a callback when it ends
- FatFs disk I/O layer, for applications that want a filesystem

1.2 What it is for

Bulk block transfer to and from an SD card from an FPGA, with a CPU that has other things to do. The hardware handles everything between "here is a command" and "the block is in memory": framing, CRCs, the tokens, the waiting, and the several places where the card is allowed to take an unbounded amount of time.

Card *protocol* — the initialisation dance, capacity classes, the CSD parse — lives in the driver, not the RTL. That split is deliberate. Initialisation is a sequence of ordinary commands with software-visible decisions in it; putting it in hardware buys nothing and makes every future card quirk an RTL change.

1.3 Status

This core has never run on a board. It has not been tested against a physical SD card. Its timing is closed in Quartus for the DE10-Lite's part, but nothing about it has been measured on hardware. What it has is:

Testbenches	3 — shifter, FIFO, full core — plus the HAL driver run against the RTL
Checks	98 in the three testbenches, 112 in the driver harness
Configurations swept	5 for the full core (<code>dma</code> , <code>pio</code> , <code>sdsc</code> , <code>tight</code> , <code>noburst</code>); 4 for the driver, plus a processor slower than the card and one with no delay between bus accesses
Bound SVA assertions	28, plus 6 cover points
Assertion fault injections	7 — each required to be caught
Platform Designer component checks	22
HAL driver build checks	4
Lint configurations	10
Documentation claims re-derived from source	see <code>doc/tools/check_facts.py</code>

The card model is derived from the specification, not from a real card. Real cards deviate from the specification in ways a model written from the same document cannot predict — that is precisely the risk that remains, and it is not a small one.

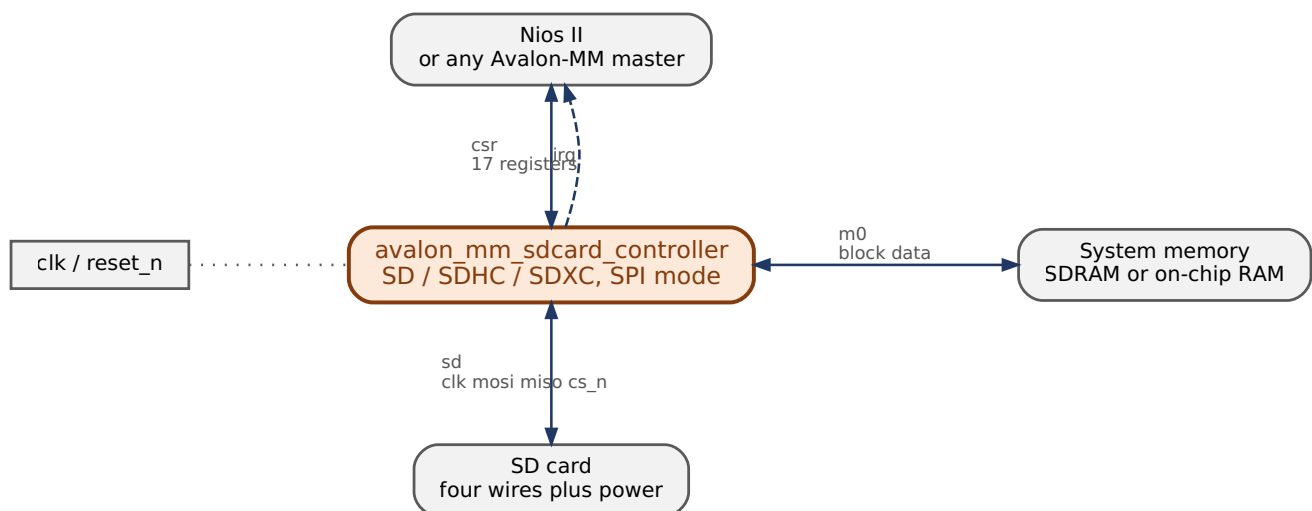
2. Getting started

2.1 Adding the component

Point Platform Designer at the directory containing `altera_avalon_mm_sdcard_controller_hw.tcl`, then add **Avalon-MM SD Card Controller (SPI)** from *Memory Interfaces and Controllers / Custom*.

2.2 Connecting it

Interface	Connect to
<code>clock</code> , <code>reset</code>	Your system clock and reset
<code>csr</code>	The CPU's data master
<code>m0</code>	The memory the data lands in — SDRAM, on-chip RAM. Only present when <code>USE_DMA = 1</code>
<code>irq</code>	The CPU's interrupt receiver
<code>sd</code>	Export to the top level, then to pins



`m0` bursts. Platform Designer inserts a burst adapter wherever it meets a slave that bursts less, so `M0_BURST_WIDTH = 1` is a supported configuration rather than a degraded one — at SPI rates the memory side is never the bottleneck.

2.3 Pin assignment

Four signals plus two optional ones. In SPI mode the SD card's pins carry different meanings from their names in native mode:

Conduit	SD card pin	Notes
<code>sd_clk</code>	CLK (pin 5)	
<code>sd_mosi</code>	CMD (pin 2)	
<code>sd_miso</code>	DAT0 (pin 7)	Needs a pull-up. The card releases the line and it must idle high
<code>sd_cs_n</code>	DAT3/CS (pin 1)	Needs a pull-up during power-up, or the card may come up in native mode
<code>sd_cd_n</code>	socket switch	Optional
<code>sd_wp_n</code>	socket switch	Optional

DAT1 and DAT2 are unused in SPI mode but should be pulled up rather than left floating.

2.4 A first transfer

The BSP has already bound the driver to the hardware: `alt_sys_init()` runs `alt_sdcard_init()` for every instance before `main()`. It has not touched the card, and nothing needs to be done before the first read.

```
#include "altera_avalon_mm_sdcard_controller.h"

extern alt_sdcard_dev sdcard;          /* from alt_sys_init.c */
alt_u32 buf[128];                     /* one 512-byte block */

if (alt_sdcard_read_blocks(&sdcard, 0, buf, 1) != ALT_SDCARD_OK)
    return -1;                         /* no card, or it refused */
```

The first block call runs the identification sequence at 400 kHz, works out the card's capacity class, reads the CSD and CID, and then raises the clock. Every call addresses the card in **512-byte blocks** regardless of class; the driver converts to a byte address for standard-capacity cards.

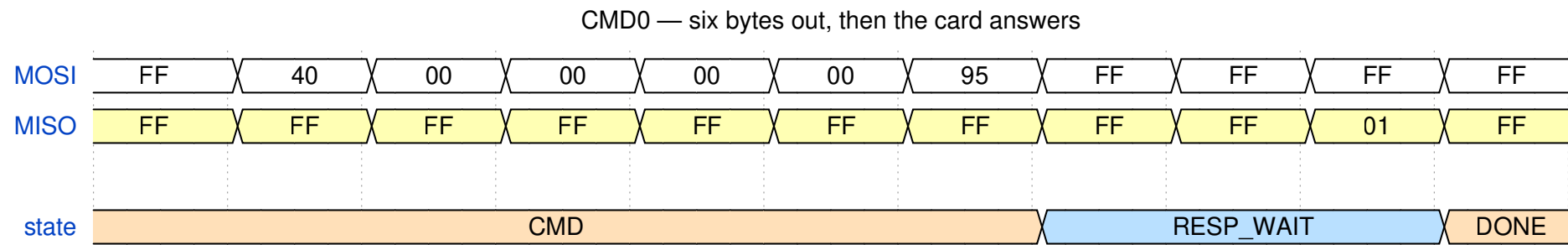
3. Functional description

3.1 The shape of an operation

The sequencer is a 20-state machine. Every operation is a command, optionally followed by a data phase:

1. **Pre-busy.** With `CS` high, the core free-runs `0xFF` until `MISO` is high. A card still programming a previous block holds the line low, and clocking it is how you find out.
2. **Command.** Six bytes: `0x40 | index`, four argument bytes, CRC7 shifted up one with the stop bit in bit 0.
3. **Response.** The core clocks `0xFF` until a byte arrives with bit 7 clear. `N_CR` is a **range** — 0 to 8 byte-times after the frame — not a fixed latency, so this is a poll and not a wait. After CMD12 the first byte is a stuff byte, which may still carry data from a stream being stopped; the core drops it unread.
4. **Data phase**, if `CMD.DATA_EN` is set. Read: wait for the `0xFE` token, take `BLK_SIZE` bytes, then a two-byte CRC16. Write: send `0xFE`, the data, the CRC16, then read the data-response token and wait out the card's busy.
5. **Done.** `IRQ_STATUS` records what happened; `ERR_INFO` records the detail if it went wrong.

A recorded command and its response, one column per byte-time:



0x95 is the CRC7 of CMD0 with a zero argument — the constant that appears in every SD initialisation routine ever written. The card answered two byte-times after the frame; the specification allows anywhere from 0 to 8, which is why the core polls rather than waiting a fixed time.

3.2 Multi-block transfers

Set `BLK_COUNT` above 1 and `CMD.MULTI`, and the core streams blocks back-to-back without software in the loop. With `CMD.AUTO_STOP` set it also terminates the stream itself: **CMD12 for a multi-block read**, the 0xFD stop-token for a multi-block write.

Note the asymmetry, which is the specification's and not this core's: the 0xFE start token covers single-block read, single-block write *and* multiple-block read. Only multiple-block **write** uses its own 0xFC, and only that form has a stop token.

3.3 Timeouts

Every wait is bounded by `TIMEOUT`, and the bound is on **time without progress**, not total duration — so a long block never trips it and a stalled one does.

`ERR_INFO[27:24]` records which phase failed. This matters more than it looks: "the card never answered" and "the card answered and then never finished" need different recovery, and nothing else distinguishes them.

3.4 Soft resets

`CTRL` carries three reset bits, and they are separate for a reason:

Bit	Resets
<code>SRST_CMD</code>	The command path only
<code>SRST_DAT</code>	The data path and the FIFO only
<code>SRST_ALL</code>	Everything except configuration

Resetting the data path after a failed transfer leaves the card identified, so recovery does not mean redoing the whole 400 kHz initialisation. The regression checks this specifically, because a "reset" that quietly loses card state is a reset nobody can use.

3.5 CRC

	Polynomial	Initial value
CRC7, command frames	$x^7 + x^3 + 1$	0
CRC16, data blocks	$x^{16} + x^{12} + x^5 + 1$ (CCITT)	0x0000

That CRC16 initial value is the trap. "CCITT" in almost every other context means an initial value of 0xFFFF; SD specifies zero. A CRC16 seeded with 0xFFFF produces plausible-looking values that never match, and the failure looks exactly like a wiring fault.

Both are computed a byte at a time, because the shifter does not pause between bytes and a bit-serial CRC would have nowhere to do its work.

CRC7 on commands is unconditional even when `USE_CRC = 0`: CMD0 and CMD8 are validated by the card whatever CMD59 says.

4. Parameters

Parameter	Default	Range	Effect
FIFO_DEPTH_BYTES	1024	512, 1024, 2048, 4096, 8192	Decouples the shifter from memory. 1024 holds two blocks, so one can be on the wire while the other moves. 512 works but leaves no overlap
M0_BURST_WIDTH	8	1–9	Maximum burst is $2^{(N-1)}$ beats. 8 gives 128 beats, exactly one 512-byte block. 1 means no bursting — supported, not degraded
CLKDIV_WIDTH	8	4–16	SPI clock is $\text{clk}/(2 \cdot \text{CLKDIV})$, set at run time. 8 bits spans $\text{clk}/2$ to $\text{clk}/510$
TIMEOUT_WIDTH	26	16–32	26 bits at 100 MHz is 0.67 s, covering the specification's 250 ms write-busy and 100 ms read-access limits with margin
MAX_BLOCK_BYTES	512	16, 64, 128, 256, 512	512 is the specification maximum, not a convention
CSR_ADDR_WIDTH	5	5–8	In words . The map occupies 17, so 5 is the minimum
ADDR_WIDTH	32	16–32	Byte address width of <code>m0</code> . Ignored when the DMA is off
USE_DMA	1	0, 1	Off removes <code>m0</code> entirely. Costs roughly 10–20% of a 100 MHz Nios II/f during a transfer and nothing in throughput
USE_CARD_DETECT	1	0, 1	Adds <code>sd_cd_n</code> / <code>sd_wp_n</code> and the insert/remove interrupts. Off reports a card always present and never write-protected
USE_CRC	1	0, 1	CRC16 on data blocks. A debugging aid, not a performance option — it costs nothing

4.1 On MAX_BLOCK_BYTES

512 bytes is fixed by Physical Layer 7.2.3 regardless of `READ_BL_LEN`, and SDHC/SDXC cards accept nothing else. Smaller values only make sense for a design that will never touch a high-capacity card, or one that only ever reads the 16-byte CSD and CID.

This is worth being explicit about because it is easy to assume `BLK_SIZE` is freely programmable. It is — on the *controller*. The **card** keeps its own block length, 512 until CMD16 changes it, and CMD16 does not change it on a high capacity card. Setting the controller to a smaller block than the card is sending desynchronises the transfer: the read fails its CRC16, the write never gets a data-response token, and the next command is swallowed as data.

4.2 On USE_DMA

Turning the DMA off is a legitimate choice for a system with no suitable memory target, or none to spare. What changes is *who sets the pace*: with no master keeping the buffer moving, software does, in both directions. A write whose next word has not arrived stops the SPI clock until it does. A read block is only clocked in once the buffer has room for all of it, so the clock stops between blocks until software has drained enough. Either way the card waits, and a processor slower than the card makes the transfer slower, not wrong.

What software must not do is stop servicing the `DATA` window altogether. The data-phase timeout bounds time **without progress**, and after `TIMEOUT` the transfer ends with `ERR_DAT_TMO`. A write abandoned in the middle of a block leaves the card waiting for the rest of it, and a failed multi-block read leaves it sending, which is why the driver issues CMD12 after any failed transfer.

Both paths are tested rather than assumed — `p10` is one of the five configurations the regression sweeps, and the full-core suite also clears `DMA_EN` at run time to starve a write and abandon a read in every one of them.

5. Registers

All registers are 32 bits, word-addressed on `csr`. Read latency is 1.

Offset	Word	Name	Access	Purpose
0x00	0	CTRL	RW	Enable, CS control, CRC, DMA, soft resets
0x04	1	STATUS	RO	Busy flags, FIFO level, card presence, error summary
0x08	2	IRQ_ENABLE	RW	Mask, one bit per source
0x0C	3	IRQ_STATUS	RW1C	Pending events and errors
0x10	4	CLKDIV	RW	SPI clock = $\text{clk} / (2 \cdot \text{CLKDIV})$
0x14	5	TIMEOUT	RW	Cycles without progress before giving up
0x18	6	CMD_ARG	RW	The command's 32-bit argument
0x1C	7	CMD	RW	Index, response type, data flags. Write starts the operation
0x20	8	RESP0	RO	R1 in [7:0]. Cleared when a command starts
0x24	9	RESP1	RO	The R3 / R7 trailer, or R2's second byte in [7:0]. Cleared when a command starts
0x28	10	BLK_SIZE	RW	Bytes per block, up to MAX_BLOCK_BYTES
0x2C	11	BLK_COUNT	RW	Blocks in this transfer
0x30	12	DMA_ADDR	RW	Byte address in system memory
0x34	13	DMA_CTRL	RW	Mode. Only contiguous (0) is defined
0x38	14	DATA	RW	PIO window into the FIFO
0x3C	15	ERR_INFO	RO	Last tokens, last R1, phase that failed
0x40	16	CORE_INFO	RO	Build-time configuration

5.1 CTRL (0x00)

Bit	Name	Meaning
0	ENABLE	Master enable. 0 idles the sequencer
1	CS_MANUAL	Software drives CS_n directly
2	CS_VALUE	The level driven when CS_MANUAL is set
3	CRC_EN	Generate and check CRC16 on data blocks
4	DMA_EN	Data phase uses m0 rather than the DATA window
5	CLK_RUN	Free-run the SPI clock with CS_n high
8	SRST_CMD	Soft reset, command path only
9	SRST_DAT	Soft reset, data path and FIFO only
10	SRST_ALL	Soft reset, everything except configuration

CS_MANUAL and CLK_RUN exist for initialisation: the specification requires at least 74 clocks with CS high before the first command, and software drives that directly rather than the sequencer inventing a pseudo-command for it.

5.2 STATUS (0x04)

Bit	Name	Meaning
0	CMD_BUSY	A command is in flight
1	DAT_BUSY	A data phase is in flight
2	DMA_BUSY	m0 has a transfer outstanding
3	CARD_BUSY	The card is holding MISO low
23:8	LEVEL	FIFO occupancy in bytes
24	CARD_PRES	From sd_cd_n , if USE_CARD_DETECT
25	CARD_WP	From sd_wp_n , if USE_CARD_DETECT
31	ERROR	Sticky OR of the error bits in IRQ_STATUS

5.3 IRQ_STATUS (0x0C) and IRQ_ENABLE (0x08)

Write 1 to clear. IRQ_ENABLE has the same layout.

Bit	Name	Meaning
0	CMD_DONE	Command and its response complete
1	DATA_DONE	The whole BLK_COUNT transfer is done
2	DMA_DONE	m0 has drained or filled the FIFO
8	ERR_CMD_TMO	No response within TIMEOUT
9	ERR_CMD_CRC	R1 reported a command CRC error
10	ERR_CMD_ILL	R1 reported an illegal command
11	ERR_DAT_TMO	No data token, or busy outlasted TIMEOUT
12	ERR_DAT_CRC	CRC16 mismatch on a read block
13	ERR_DAT_TOKEN	The card sent a data error token
14	ERR_WRITE	The data-response token rejected the block
15	ERR_DMA	m0 returned an error response
16	CARD_INSERT	sd_cd_n went low
17	CARD_REMOVE	sd_cd_n went high
18	ERR_PIO	A DATA access the buffer could not serve

ERR_PIO is the one to know about if you use the DATA window. A write when the buffer is full, or a read when it is empty, is refused — and refusing it is the only correct thing to do, since there is no room for the word or no word to give. Before this bit existed the refusal was also silent, so the only evidence was wrong bytes at the far end of the transfer. It covers using the window in the wrong direction for the transfer in flight too, which is the same mistake.

Bits 16 and 17 are events, not errors. STATUS.ERROR and the driver's error mask cover bits 8–15 and 18 only. A mask that included the card-detect bits made STATUS.ERROR assert because a card was fitted — from the first cycle after reset, since an already-present card reads as an insertion.

An interrupt handler must read IRQ_STATUS rather than infer the cause from the pin. During a data phase DMA_DONE raises the line repeatedly as bursts retire, so the pin being high says only that something happened.

5.4 CMD (0x1C)

Bit	Name	Meaning
5:0	INDEX	Command index
7:6	RESP	0 = R1, 1 = R1b, 2 = R2, 3 = R3/R7
8	DATA_EN	This command has a data phase
9	DATA_DIR	0 = card → host, 1 = host → card
10	MULTI	Stream BLK_COUNT blocks in hardware
11	AUTO_STOP	Terminate the stream without software
31	START	Write 1 to launch. Reads back as busy

The write is ignored while the sequencer is busy. That is correct — a second command must not corrupt a transfer in flight — but it means a driver that writes without checking loses the command silently, and polling afterwards does not catch it: busy is already clear, so the poll returns immediately for a command that never happened. Check `STATUS.CMD_BUSY` before writing, and confirm it goes high afterwards.

5.5 ERR_INFO (0x3C)

Bits	Meaning
7:0	Last data-response token
15:8	Last R1 byte received
23:16	Last data error token
27:24	The phase that timed out

The data-response token's shape is `xxx0sss1` — five bits of meaning in eight. Mask with `0x1F` before comparing. `0x05` is accepted (`sss = 010`), `0x0B` is a CRC error (`101`), `0x0D` is a write error (`110`).

6. Signals

6.1 Clock and reset

Signal	Direction	Width	Notes
clk	in	1	System clock. One domain — there is no SPI clock domain
reset_n	in	1	Active low, synchronous

The SPI clock is generated by strobes inside the same domain rather than as a second clock, so there is no clock-domain crossing anywhere in this core.

6.2 `csr` — Avalon-MM slave

Signal	Direction	Width
csr_address	in	CSR_ADDR_WIDTH (words)
csr_read	in	1
csr_write	in	1
csr_writedata	in	32
csr_byteenable	in	4
csr_readdata	out	32

6.3 `m0` — Avalon-MM master (`USE_DMA = 1`)

Signal	Direction	Width
m0_address	out	ADDR_WIDTH (bytes)
m0_read, m0_write	out	1
m0_writedata	out	32
m0_byteenable	out	4
m0_burstcount	out	M0_BURST_WIDTH
m0_waitrequest	in	1
m0_readdata	in	32
m0_readdatavalid	in	1
m0_response	in	2

`m0_byteenable` is always `0xF`. That is not laziness: Avalon permits the interconnect to suppress a read with all byteenables clear, which would hang the transfer with no error reported anywhere. An assertion enforces it, and `check_assertions_fire.sh` injects the fault to prove the assertion is alive.

6.4 sd — conduit

Signal	Direction	SD pin
sd_clk	out	CLK
sd_mosi	out	CMD
sd_miso	in	DAT0
sd_cs_n	out	DAT3/CS
sd_cd_n	in	socket switch (USE_CARD_DETECT)
sd_wp_n	in	socket switch (USE_CARD_DETECT)

7. Software

The HAL driver is at `HAL/src/altera_avalon_mm_sdcard_controller.c`, with the API in `HAL/inc/altera_avalon_mm_sdcard_controller.h`. The BSP finds it by itself, constructs every instance in `alt_sys_init.c` and calls `alt_sdcard_init()` before `main()`.

Function	Purpose
<code>alt_sdcard_init</code>	Bind to the hardware: check <code>CORE_INFO</code> , learn the build, register the ISR. Called by the BSP
<code>alt_sdcard_read_blocks</code>	Read blocks into memory, identifying the card first if none is
<code>alt_sdcard_write_blocks</code>	Write blocks from memory, likewise
<code>alt_sdcard_read_blocks_start</code> , <code>alt_sdcard_write_blocks_start</code>	The same with the DMA, returning once the transfer is under way — §7.5
<code>alt_sdcard_transfer_status</code>	<code>ALT_SDCARD_ERR_BUSY</code> , or the last non-blocking transfer's result; finishes one that has ended
<code>alt_sdcard_probe</code>	Identify the card now: 400 kHz sequence, capacity class, CSD and CID, then raise the clock
<code>alt_sdcard_check</code>	Has the socket changed? Reads the switch and the latched events; sends nothing
<code>alt_sdcard_poll</code>	<code>alt_sdcard_check</code> , then CMD13 — is the card still answering?
<code>alt_sdcard_generation</code>	Counts identifications; changes whenever the card behind the block calls might have
<code>alt_sdcard_block_count</code>	Card capacity in 512-byte blocks, parsed from the CSD
<code>alt_sdcard_command</code>	Issue an arbitrary command. <code>ALT_SDCARD_RESP_AUTO</code> takes the response format from the index
<code>alt_sdcard_present</code> , <code>alt_sdcard_write_protected</code>	The socket switches; 1 and 0 on a core built without them
<code>alt_sdcard_reset_datapath</code>	Clear a wedged data path without losing the identification
<code>alt_sdcard_set_event_handler</code>	A callback from the ISR. Installing one enables the card-detect interrupts
<code>alt_sdcard_instance</code>	The n-th controller, for code that is not handed a device

Every other call busy-waits, polling `STATUS` for completion. Under an RTOS that holds the calling task for the transfer — about 170 μ s a block at 25 MHz — which suits a filesystem task and not an interrupt handler. The `_start` calls are the exception.

7.1 Capacity, and why the CSD parse is separately tested

`alt_sdcard_block_count()` parses capacity out of the CSD, and the arithmetic is **different for the two structure versions** — not a field that moved. Reading a v2 card with the v1 formula yields a plausible number that is wrong by orders of magnitude, which is the kind of bug that survives casual testing and then corrupts a filesystem.

So it is unit-tested directly, against the exact CSD bytes the card model returns for each class, in `verification/check_driver_builds.sh`, as well as through a real identification in the driver harness.

7.2 Card changes

A card can be pulled and another fitted between any two calls, and the new one may be a different capacity class, which changes the unit a block number is sent in. So the driver never carries an identification across a change it can see:

- **With a card-detect switch**, the switch and the latched `CARD_INSERT` and `CARD_REMOVE` events are checked at the start of every call. The events are what catch a fast swap, which the switch alone reads as "present" both times. An interrupt handler may take them first; the driver still sees them.
- **Without one**, there is nothing to read. But a card inserted behind the driver's back is still in SD mode and cannot answer, so it shows up as a timeout, and a card that times out is forgotten. `alt_sdcard_poll()` finds out sooner, at the cost of one command.

The call that discovers a change fails with `ALT_SDCARD_ERR_CHANGED`, or `ALT_SDCARD_ERR_NO_CARD` if the socket is empty, and the next call identifies whatever is there. Failing once is deliberate: a caller holding anything it learned from the old card must not have its next write land on the new one.

7.3 Retries

A CRC error means the link corrupted something, not that the card refused. A command whose CRC fails is not executed; a read can be repeated; and a block the card rejected on CRC (data response `0x0B`) was never programmed. Those are retried inside the call, `dev->retries` times — 2 by default. Nothing else is retried: a timeout, an error the card reported, or a write it refused for any other reason (`0x0D`) goes straight back to the caller.

`alt_sdcard_command()` itself never retries, because after CMD55 repeating a failed application command would send the ordinary command with the same index.

After any failed transfer, retried or not, the driver resets the data path and sends CMD12 before anything else. Resetting the controller does not touch the card: a card in a multi-block read goes on sending until CMD12, whatever went wrong at the host, and one left part-way through a write waits for the rest. Either answers the next command as illegal. A card that had already finished answers the CMD12 as illegal instead, and nothing is lost but the command.

7.4 Buffers and PIO

Any buffer works. With the DMA a word-aligned buffer moves in one multi-block stream, and a misaligned one — which the DMA cannot address — goes a block at a time through a bounce buffer on the stack. Either way it must be memory the DMA master can reach. A transfer longer than `BLK_COUNT`'s 16 bits is issued in pieces.

With `USE_DMA = 0` the driver moves every word through the `DATA` window while the transfer runs. It has to: on a write the sequencer reaches the data phase about ten byte-times after the command goes out, and a buffer still empty then stalls the shifter. A read can be drained as slowly as the processor needs — the core holds the clock until the buffer has room for the next block — but not abandoned: after `TIMEOUT` with no room it gives up with `ERR_DAT_TMO`.

The loop reads `STATUS` once per batch. `LEVEL` says how many bytes are waiting, so it takes every whole word there — or, writing, fills every word of room — with `DATA` accesses back to back, then looks again. Where the processor is slower than the card that is half the bus traffic of reading `STATUS` before every word, and so half the transfer time.

7.5 Non-blocking transfers

With the DMA, a transfer needs the processor to start it — six register writes — and to read its result. The blocking calls spend the time in between reading `STATUS` as well. `alt_sdcard_read_blocks_start()` and `alt_sdcard_write_blocks_start()` take the same arguments plus a callback and a context pointer, and return as soon as the transfer is under way:

```
static void sd_done(alt_sdcard_dev *dev, int result, void *context)
{
    /* result: what alt_sdcard_read_blocks() would have returned */
}

r = alt_sdcard_read_blocks_start(&sdcard, block, buf, n, sd_done, ctx);
/* ... other work ... */
r = alt_sdcard_transfer_status(&sdcard); /* ALT_SDCARD_ERR_BUSY until it ends */
```

- **How the end is noticed.** With the controller's interrupt connected, the driver adds `DATA_DONE` to `IRQ_ENABLE` while the transfer runs, and `CMD_DONE` while it stops the card after a failure, and removes again what it added. Its ISR takes each step and calls the callback. Without one (`irq` is -1 in `system.h`) nothing moves until `alt_sdcard_transfer_status()` is called, which takes the step if the step has ended — so call it from the main loop. The callback runs once, in whichever context noticed the end, and may be 0.
- **What happens on the way** is what the blocking call does: CRC errors retried `dev->retries` times, CMD12 after a failure, a card that times out forgotten, a transfer longer than `BLK_COUNT` issued in pieces. The callback's result is the one the blocking call would have returned.
- **What is refused.** `ALT_SDCARD_ERR_PARAM` in a build without the DMA, where the processor moves every word and there is nothing to hand back, and for a buffer that is not word-aligned, which the blocking calls send through a bounce buffer on their stack. Otherwise the start returns what the blocking call returns before its first command: a change of card, no card, write protection, a range past the end of the card. If no card is identified, the start call identifies one first, and that part blocks.
- **While it runs**, every call that talks to the card — the block calls, a second start, `alt_sdcard_command`, `alt_sdcard_probe`, `alt_sdcard_check`, `alt_sdcard_poll` — returns `ALT_SDCARD_ERR_BUSY`. The buffer must stay valid until it ends. `alt_sdcard_reset_datapath()` abandons it: the status then reads `ALT_SDCARD_ERR_TIMEOUT`, and the callback is not called.
- **Chaining.** The callback may start the next transfer. Called from `alt_sdcard_transfer_status()`, that call still returns the result of the one that ended. Check `result` before starting another from the ISR: after a timeout the card is forgotten, and the start would identify it there, which takes hundreds of milliseconds.
- **An event handler** installed with `alt_sdcard_set_event_handler()` sees the `DATA_DONE` and `CMD_DONE` interrupts too, before the driver takes its step. Code of your own that changes `IRQ_ENABLE` while a transfer runs must hold interrupts off from the read to the write, as the driver does: an interrupt in between can add a bit the transfer needs, and the write takes it away.

In the driver harness, eight blocks cost the processor 25 bus accesses from start to callback in one `BLK_COUNT`, and 59 split into three.

7.6 FatFs

`software/fatfs/diskio_altera_sdcard.c` implements FatFs's disk I/O functions — `disk_status`, `disk_initialize`, `disk_read`, `disk_write`, `disk_ioctl` — on the block API. FatFs is not included (R0.14 or later). Add the file to the application in place of FatFs's template `diskio.c`; the BSP does not build it. Drive `n` is the `n`-th controller instance, and if FatFs has other drives, `SDCARD_DISKIO_NAME` renames the five functions for your own dispatcher.

It ties a mounted volume to the card it was mounted from. Transfers are refused with `RES_NOTRDY` on anything but the identification `disk_initialize` accepted, and `disk_status` reports `STA_NOINIT` after a change, which makes FatFs drop the volume and mount afresh; an open file on the old card fails with `FR_INVALID_OBJECT`. On a socket with no switch, `disk_status` asks the card with CMD13, because FatFs answers enough from its cache that a failed transfer comes too late.

`GET_BLOCK_SIZE` reports 1, FatFs's "unknown", and `CTRL_TRIM` is not implemented.

8. Performance

8.1 Measured

16,696 SPI clocks to move 2,048 bytes, against a theoretical floor of 16,384 — **98.1% of line rate**. At a 25 MHz SPI clock that is 3.07 MB/s of a possible 3.125.

The regression **asserts** this cycle count rather than reporting it. A performance number nobody checks is a number that silently regresses.

8.2 Where it comes from

The shifter runs continuously with a one-deep prefetch, so the SPI clock does not pause between bytes. The remaining 1.9% is the protocol's own overhead — command frames, response polling, tokens, CRC bytes — not idle clocks.

8.3 What the DMA is and is not for

Not throughput. At SPI rates the memory side is never the bottleneck, and `M0_BURST_WIDTH = 1` measures the same as `8`. What the DMA buys is **CPU time** and **immunity to interrupt latency**: without it the CPU must service the `DATA` window for the whole of every transfer, and the transfer goes no faster than the CPU does. The blocking calls spend that time anyway, reading `STATUS` until the transfer ends; the calls in §7.5 are the ones that give it back.

9. Verification

Everything that can be checked in software alone:

```
./verification/run_all.sh
```

Suite	What it does
Lint	Verilator <code>-Wall</code> across 10 parameter configurations
Simulation	3 testbenches, the full-core one in 5 configurations, and the HAL driver run against the RTL in 4 builds
Platform Designer	<code>hw.tcl</code> executed against stubbed Qsys commands — 22 checks
HAL driver	Compiled against stubbed Nios II headers, the FatFs glue with it, plus the CSD parse unit-tested
Assertions	7 faults injected, each required to be caught by the assertion meant to catch it
Facts	Every number in the documentation re-derived from source
CRC vectors	The polynomials checked against an independent Python model

9.1 The configuration sweep

The five configurations are not cosmetic variations:

Configuration	What only it reaches
<code>dma</code>	The reference case
<code>pio</code>	No master at all; software moves every word in every transfer, where the other builds use the <code>DATA</code> window only when a test clears <code>DMA_EN</code>
<code>sdsc</code>	Byte addressing. On an SDHC card the block-to-address conversion is the identity, so this is the only place it executes
<code>tight</code>	One block of buffer, so the data path refills mid-transfer
<code>noburst</code>	Single-beat Avalon transactions throughout

Adding this sweep immediately found a real defect: neither `S_RD_DATA` nor `S_WR_DATA` checked the timeout, so a data phase starved of data hung the core until a soft reset. With a master attached that cannot happen, which is exactly why it survived until the PIO configuration was run.

9.2 Assertions, and proving they are alive

28 bound SVA assertions and 6 cover points, in `tb/avalon_mm_sdcard_controller_sva.sv`.

They check invariants rather than results. That suits this core: most of its bugs were invisible on the wire — a byte queued twice, a CRC fed seven of eight bits, a transfer declared complete with a byte still in the shifter — and each one breaks an invariant you can state in a line.

A passing assertion proves nothing by itself, so `verification/check_assertions_fire.sh` injects seven faults into scratch copies of the RTL and requires each to be caught by the assertion meant to catch it.

The first fault it tried turned out to be **unreachable**. `S_PRE_BUSY` clocks `0xFF` before every command, so the shifter is always already running when a sending state begins, and the `!hold_v` term in its idle output can never fire. The script records that rather than swapping in a fault that worked.

9.3 The driver against the RTL

Everything above drives the registers from SystemVerilog written to do what the driver does. `tb/driver/` runs the driver itself: `HAL/src` compiled with gcc, unmodified, linked into the Verilator model, with a small C++ harness standing in for the processor — each register access is one Avalon-MM transfer, and nothing else moves simulated time. Two cards, one of each capacity class, can be swapped in the socket between calls or in the middle of one. It runs with and without the DMA and a card-detect switch, on a processor more than twice as slow as the card and on one with no delay between bus accesses, 112 checks each time, and it calls the FatFs glue the way FatFs does.

Its first run found two faults every other suite had passed. The driver's power-up clocks were timed by a CPU loop rather than by anything the bus can count. And the sequencer opened the response window one byte early at 25 MHz: a card answering at the maximum `N_CR` of 8 timed out, and a CMD12 stopping a read could take the card's data for its response. Both are fixed and each has a test that fails without the fix; the README's verification section has the detail.

Its "slow processor" was not slow: 82 clock cycles per word read through `DATA`, a `STATUS` read and a `DATA` read each, against the card's 128. Run slower than the card, it showed that a multi-block read kept clocking into a full buffer and lost the bytes that did not fit, while the CRC — checked on the wire — passed and the driver returned `ALT_SDCARD_OK`. A memory stalling the DMA on a block's last word lost the next block's DMA start the same quietly. Both are fixed by admitting a read block only once the buffer has room for it and the DMA is free, and the README's section *A read the host cannot keep up with* has the detail. The slow run now takes 301 clock cycles per word, and must see blocks held, since the PIO loop reads `STATUS` once per batch of words rather than before every word — which a further check holds it to.

Adding the non-blocking calls found two faults in the driver. A `STATUS` read in the cycle after a `CMD` write finds the core idle — it shows the command as busy two cycles later — and every wait in the driver took idle for finished, so on a processor with no delay between accesses it could not identify a card; a wait now also requires `CMD_DONE` or `DATA_DONE` latched. And a failed multi-block read left the card sending, because only a failed write was followed by CMD12, so a CRC error's retry was refused and the card was unusable after it. A check written for a third found it: installing an event handler while a transfer recovered could write over the `CMD_DONE` the interrupt handler had just enabled, and the transfer's callback never came. The README's *What the non-blocking calls found* has the detail.

9.4 Questa

`simulation/questa/run_sim.tcl` runs the same sweep with coverage and non-vacuity reporting.

It has been run, against Questa 2024.1, and all seven configurations pass with every assertion present and passing non-vacuously somewhere in the sweep, and all six cover directives present and reached. Treating that first run as part of the work rather than a formality was the right call: it found four faults, and the worst was that **none of the assertions had ever been running**. The binds sit at compilation-unit scope, so without `-mfcu -cuname` the SVA modules compiled and none of them elaborated — seven configurations reporting "no assertion failures" because there were no assertions. The verdict now requires each assertion by name before it may report a pass.

It also found RTL that `vopt` rejected outright and Verilator linted clean, one assertion whose consequent was the literal `1'b1` and which therefore could never fail, and a memory model that never backpressured a read command.

The sequencer now reaches all 20 of its states and 41 of its 58 transitions. The 17 remaining are accounted for: 16 are the single `if (srst)` statement counted once per source state, of which the three that matter are tested; and one is the defensive timeout in `S_WR_CRC`, which cannot fire as the sequencer is wired, since that state cannot be starved of a byte. The timeout in `S_RD_DATA` was listed beside it until a read block could be held for the host; a read that nobody drains now reaches it. See the README's verification section for the reasoning.

10. Limitations and known gaps

- **Never run on a board.** No physical card and no board example. This is the largest gap and no amount of simulation closes it.

Timing closure is no longer part of it. The core synthesises, fits and meets a 100 MHz clock on the DE10-Lite's 10M50DAF484C7G in Quartus Prime 18.1 — Fmax 109.51 MHz at the slow 85 °C corner, +0.868 ns of slack, 1774 logic cells and one M9K. That is checked by `verification/check_synthesis.sh` on every run, and the component itself is loaded into real Platform Designer by `verification/check_qsys.sh`. What remains unproven is a transfer to an actual card. - **The card model is derived from the specification**, so it cannot reproduce the ways real cards deviate from it. - **SPI mode only.** No 1-bit or 4-bit native SD, and no path to it in this design — SDHCI, the obvious standard to adopt for native mode, contains zero occurrences of "SPI" and cannot be retrofitted here. - **No CMD12 for multi-block writes** beyond the 0xFD stop token, which is what the specification requires; there is no `AUTO_STOP` equivalent issuing a separate command on the write side. - **Non-blocking transfers need the DMA** and a word-aligned buffer, and there is one at a time per controller. Without the DMA every call blocks. - **DMA_CTRL defines only contiguous mode.** The field exists so a descriptor mode could be added as a reserved encoding rather than an ABI break. - **No erase, no lock/unlock, no SDIO.** The FatFs glue does not implement `CTRL_TRIM` for the same reason. - **Without a card-detect switch, a card change costs a timeout** to discover — one command's `N_CR` bound when the new card is in SD mode, which is the only way a card fitted behind the driver's back can be.

11. Revision history

Version	Change
v1.0	First issue.